

CopySight × Soneium Blockchain Module

Product & Development Scope for WBS Preparation

CopySight × Soneium

1. Objective

Build a blockchain integration module connecting the existing **CopySight IP Validation Layer** with **Soneium**.

The module should provide two core capabilities:

1. Proof of Creation / Provenance

- allow a creator to register an original asset;
- analyze the asset through CopySight;
- create an immutable blockchain record on Soneium;
- return the blockchain provenance reference to CopySight.

2. IP Validation Before Registration

- analyze assets before they are registered/minted through the integrated Soneium workflow;
- identify potentially protected IP;
- return a standardized RISKY / NOT_RISKY decision;
- allow the integrating Soneium application to flag or stop suspicious registrations.

| The CopySight detection engine is not in scope for redevelopment.

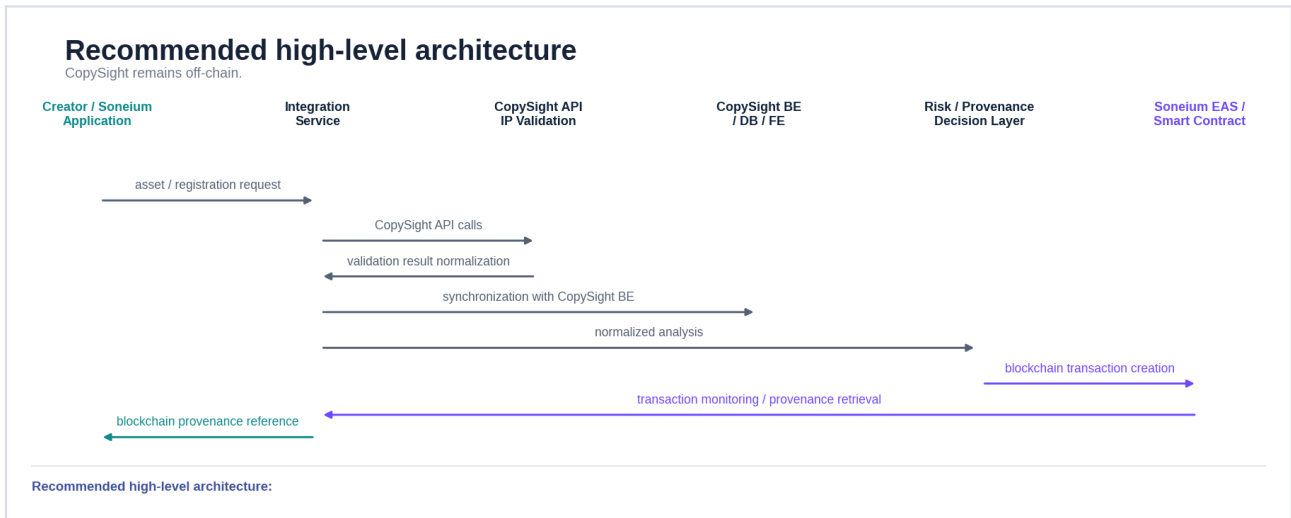
CopySight already exposes an API for image IP detection. Its current public endpoint detects IP categories including trademarks, brand/iconic designs, characters, celebrities/famous people, and art/artists, and returns detected entities, ownership information, bounding boxes and similarity scores.

The scope is therefore primarily:

CopySight API → Integration Backend → Soneium → CopySight FE/BE

2. Target Architecture

Recommended high-level architecture:



Core architectural principle

CopySight remains off-chain.

A Solidity contract cannot make a normal HTTP request to:

```
POST https://api.copsight.ai/v1/verify
```

Therefore, a new **CopySight–Soneium Integration Service** must sit between the blockchain and CopySight API.

It will be responsible for:

- asset hashing;
- CopySight API calls;
- validation result normalization;
- registration authorization;
- blockchain transaction creation;
- transaction monitoring;
- provenance retrieval;
- synchronization with CopySight BE.

3. A. Soneium Sandbox Integration

Goal

Establish a working development integration between CopySight and **Soneium Minato testnet** before mainnet deployment.

Soneium Minato is the current Soneium test network, with chain ID 1946, while Soneium Mainnet uses chain ID 1868.

Scope

A1. Development environment

Configure:

- Soneium Minato RPC;
- development wallet(s);
- deployment wallet;
- CopySight blockchain signer;
- test ETH/faucet access;
- contract deployment pipeline;
- blockchain client;
- environment configuration.

Recommended client implementation:

- Viem or ethers.js;
- TypeScript where compatible with the existing CopySight backend.

Soneium provides documented Viem support for reading, writing and interacting with contracts on Minato.

A2. RPC architecture

Production architecture must use a suitable Soneium RPC/provider rather than relying on free public RPC endpoints for high-volume application traffic. Soneium explicitly notes that its free endpoints are rate-limited and are not intended for production dApp/indexer workloads.

The implementation must also respect Soneium's current write-path requirements and use an approved transaction submission path rather than self-hosted proxy/hopping infrastructure.

A3. Team communication

Technical integration communication will happen directly through the shared **Telegram technical channel** between:

- CopySight team;
- Soneium technical team;
- Blockchain Module team.

Telegram should be used for:

- RPC/infrastructure questions;
- testnet deployment coordination;
- Soneium-specific integration issues;
- contract review;
- mainnet deployment coordination.

Architecture decisions and confirmed parameters should additionally be captured in the project documentation/repository rather than existing only in Telegram.

4. B. Proof of Creation

Goal

Allow creators to establish an immutable timestamped provenance record for newly created assets.

The blockchain record provides evidence that:

a particular account presented a particular digital asset to CopySight at or before a specific blockchain timestamp.

It should not claim that the blockchain itself legally establishes copyright ownership. CopySight itself describes its service as a technical similarity-detection tool rather than legal analysis.

4.1 User Flow

Step 1 — Creator submits asset

Creator submits:

- image;
- creator/account ID;
- connected wallet address where applicable;
- optional title;
- optional asset metadata.

Step 2 — Generate canonical asset hash

Integration Backend calculates a deterministic hash of the original asset.

Example:

```
assetHash = keccak256(originalFileBytes)
```

The actual original image should **not** be stored directly inside the smart contract.

Step 3 — CopySight analysis

Integration Backend sends the asset to CopySight.

Current API:

```
POST /v1/verify
```

CopySight returns its existing IP analysis.

Step 4 — Normalize analysis

Integration Backend converts the CopySight result into a blockchain-compatible record.

Example:

```
assetHash  
creator  
assetType  
validationStatus  
maxSimilarity  
analysisHash  
analysisVersion
```

Step 5 — Register provenance on Soneium

Create an on-chain provenance attestation.

Step 6 — Return blockchain reference

Return:

```
{
  "status": "registered",
  "chain": "soneium",
  "network": "minato",
  "assetHash": "...",
  "attestationUID": "...",
  "transactionHash": "...",
  "blockNumber": "...",
  "timestamp": "..."
}
```

Step 7 — CopySight persistence

CopySight BE stores references necessary for fast retrieval:

- asset ID;
- Soneium network;
- attestation UID;
- transaction hash;
- asset hash;
- status.

CopySight FE can then display:

```
Protected / Provenance Registered
```

and provide the blockchain reference.

5. Recommended On-Chain Architecture

Use Soneium EAS as the primary provenance primitive

Soneium already deploys:

- SchemaRegistry — 0x4200000000000000000000000000000000000020
- EAS — 0x420021

on both Soneium Mainnet and Minato.

EAS is particularly suitable because:

- every attestation receives a unique UID;
- records contain the attester;
- records contain blockchain timestamps;
- custom schemas can be created;
- attestations can reference previous attestations using refUID;
- records can be retrieved programmatically.

Recommendation

| Do not build a large custom IP registry contract for MVP.

Use:

1. Soneium EAS;
2. CopySight-specific schemas;
3. optionally a small CopySightResolver smart contract.

6. Proposed Creation Provenance Schema

Final Solidity/EAS field types are to be confirmed by the architect and Smart Contract Developer.

Conceptually:

```
CopySightCreationProof
bytes32 assetHash
string assetType
bytes32 analysisHash
uint8 riskLevel
uint16 maxSimilarity
uint32 analysisVersion
```

EAS itself already provides:

- attester;
- recipient;
- timestamp;
- UID;
- reference UID;
- revocation status.

Therefore these values should not be duplicated unnecessarily inside the schema.

Roles

Attester

CopySight-controlled blockchain signer/service.

Recipient

Creator's Soneium/EVM wallet.

Result

The EAS UID becomes the canonical:

CopySight Provenance ID

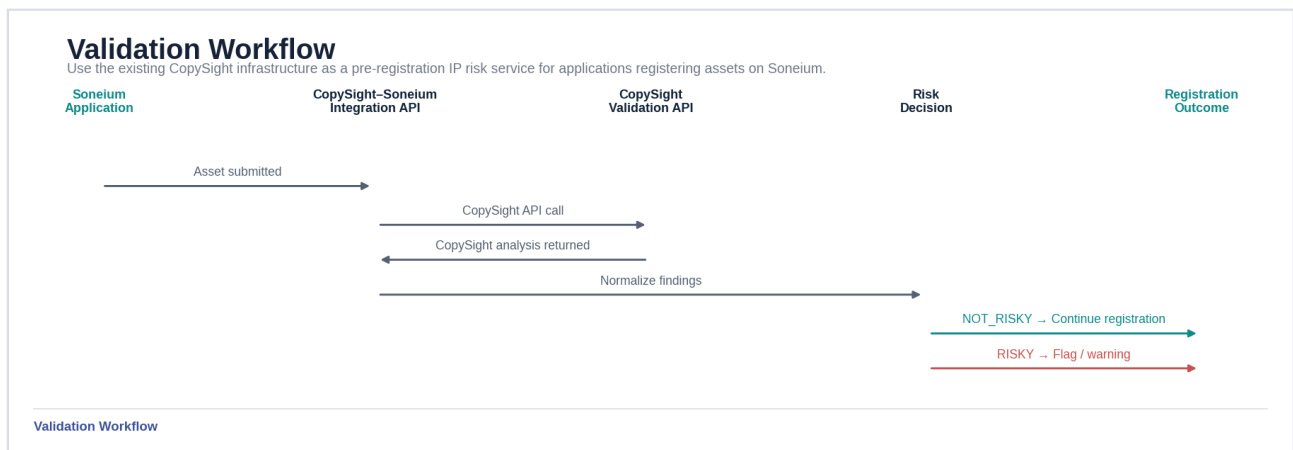
7. C. IP Validation Layer

Goal

Use the existing CopySight infrastructure as a **pre-registration IP risk service** for applications registering assets on Soneium.

This is primarily an integration task.

8. Validation Workflow



Step 1

User attempts to submit/register an asset through the integrated Soneium application.

Step 2

Application sends the asset to the CopySight-Soneium Integration API.

Step 3

Integration Backend calls CopySight.

Current CopySight results can include:

- detected IP;
- category;
- IP name;
- creator/author;
- current owner;
- similarity;
- location/bounding box;
- face detection.

Step 4

Integration Backend generates standardized risk status.

For MVP:

```

NOT_RISKY
RISKY
ERROR
  
```

Potential future extension:

```

CLEAR
LOW
MEDIUM
HIGH
REQUIRES_LICENSE
  
```

Step 5

If NOT_RISKY:

Validation passed

- registration allowed
- optional validation attestation
- Proof of Creation registration

Step 6

If RISKY:

FE displays:

This asset appears to contain protected IP.
Do you own or license the detected content?

For MVP:

- display warning;
- expose detected risk;
- return RISKY;
- allow the integrating Soneium application to decide whether registration should be blocked.

9. Important MVP Decision: Risk Score

The public CopySight API currently exposes **similarity scores per detected IP**, rather than one documented global `originality_score`.

Therefore the Integration Layer needs one of two implementations.

Option A — Recommended for MVP

Normalize existing CopySight results.

Example:

`detected_ips = []`

→ NOT_RISKY

`detected_ips > 0` and `similarity >= configured threshold`

→ RISKY

Threshold must be configuration rather than hard-coded Solidity logic.

Option B

If the CopySight internal backend already exposes a proprietary:

`originalityScore`

`riskScore`

`validationStatus`

use that field instead.

This should be confirmed between the CopySight BE team and architect before WBS estimation.

10. Validation Attestation

Optionally create a second EAS schema:

```
CopySightIPValidation
bytes32 assetHash
bool risky
uint16 maxSimilarity
bytes32 analysisHash
uint32 analysisVersion
```

The attestation proves:

CopySight evaluated asset X using analysis version Y and produced validation result Z.

It does **not** need to expose the complete raw CopySight report publicly.

Full detection details can remain available through CopySight while:

analysisHash = hash(canonical CopySight result)

anchors its integrity on Soneium.

11. CopySight–Soneium Integration Backend

This is the largest new software component.

Required capabilities

11.1 Asset hashing

- deterministic hashing;
- consistent handling of input files;
- duplicate detection.

11.2 CopySight API Adapter

Implement reusable service:

```
validateAsset(file)
```

responsible for:

- API authentication;
- multipart upload;
- parsing responses;
- timeouts;
- retries;
- rate-limit handling;
- error normalization.

The existing CopySight API documents 400, 401 and 429 response conditions that should explicitly be handled.

11.3 Risk Decision Engine

Responsible only for **normalizing CopySight output**, not performing its own AI detection.

Example:



Thresholds/configuration must be adjustable without redeploying the smart contract.

11.4 Blockchain Adapter

Functions:

```
registerCreation()  
registerValidation()  
getProvenance()  
getValidation()  
verifyAttestation()
```

11.5 Transaction Manager

Handle:

- nonce management;
- signing;
- transaction submission;
- retries;
- transaction confirmation;
- failed transaction recovery;
- RPC errors;
- idempotency.

11.6 Blockchain Sync

Track:

```
PENDING  
SUBMITTED  
CONFIRMED  
FAILED
```

and synchronize the result with CopySight BE.

12. Smart Contract Scope

Required

SC-1 — EAS schema definition

Design and register:

1. CopySightCreationProof
2. CopySightIPValidation

SC-2 — CopySight Resolver

Recommended lightweight contract:

CopySightResolver

Responsibilities:

- restrict valid CopySight attestations to authorized attester address(es);
- reject malformed attestation payloads where applicable;
- allow authorized signer rotation;
- expose necessary admin controls.

SC-3 — Deployment

Deploy/configure on:

1. Soneium Minato;
2. Soneium Mainnet after acceptance.

SC-4 — Tests

Test:

- valid attestations;
- unauthorized attester;
- malformed values;
- signer rotation;
- duplicate assets;
- revocation behavior where enabled;
- schema compatibility;
- upgrade/admin permissions.

13. CopySight Backend Scope

Existing CopySight BE capacity can be reused.

Required changes:

BE-1

Expose/confirm internal validation interface.

BE-2

Store blockchain properties:

chain

network

`assetHash`

`attestationUID`

`transactionHash`

`registrationStatus`

BE-3

Create provenance retrieval endpoints.

Example:

`GET /assets/{assetId}/provenance`

BE-4

Expose blockchain status to CopySight FE.

BE-5

Provide asset → blockchain mapping.

BE-6

Provide Integration Service authentication.

14. CopySight Frontend Scope

Reuse current CopySight frontend where possible.

Minimum UI additions:

FE-1 — Registration action

Register on Soneium

FE-2 — Blockchain status

Analyzing

Registering

Registered

Failed

FE-3 — Provenance information

Display:

- creation timestamp;
- creator/wallet;
- CopySight validation;
- Soneium network;
- Attestation UID;
- transaction reference.

FE-4 — IP warning

For risky assets:

Potential protected IP detected.

Display existing CopySight findings where appropriate.

FE-5 — Verification view

Allow a user to retrieve/verify an existing Soneium provenance record.

15. Asset Types

MVP

Supported:

- images;
- GIFs if required.

CopySight's public API currently documents image/GIF support.

Extension-ready, but not MVP

Architecture should allow:

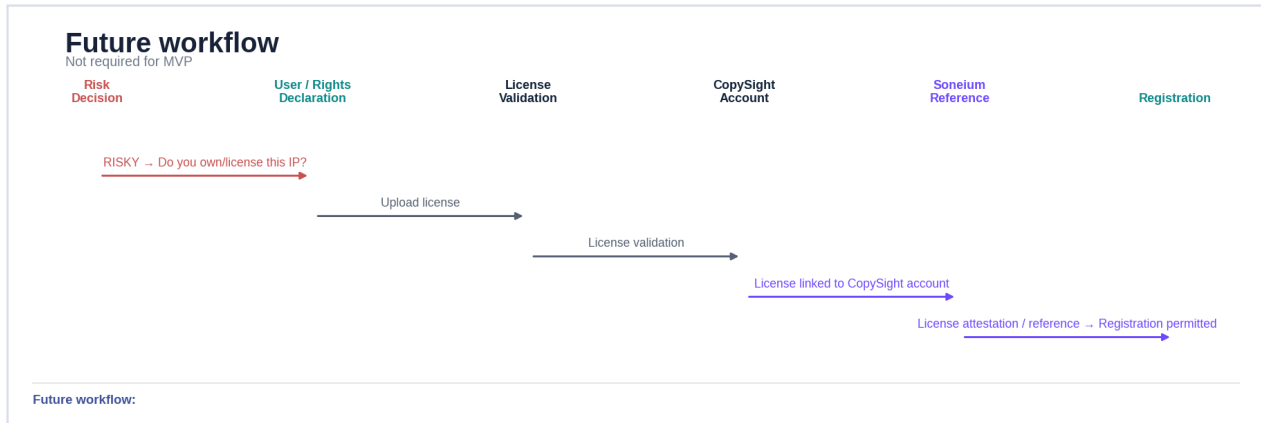
- video;
- 3D assets;
- audio;
- animation;
- other creative asset types.

However, these should **not** be included in the initial development estimate unless the corresponding CopySight validation API is confirmed.

16. Licensing Workflow

Not required for MVP

Future workflow:



Potential future EAS schema:

CopySightLicenseReference

| This is explicitly outside the short-term implementation.

17. Privacy / Data Storage Rules

Do not store:

- original image bytes;
- personal information;
- license documents;
- private account information

directly on Soneium.

On-chain data should primarily contain:

- hashes;
- wallet/account references where required;
- classifications;
- version numbers;
- timestamps;
- attestation relationships.

This is particularly important because on-chain EAS attestations are publicly available and directly readable from the blockchain.

CopySight's current policy also states that submitted content is not retained beyond what is required for processing and that AI inputs/outputs are handled under its zero-retention approach, so the blockchain design should avoid accidentally changing this property by publishing source content on-chain.

18. Operational / Security Requirements

The WBS must include:

- API key secret management;
- blockchain signer key management;
- signer rotation;
- RPC authentication;
- environment separation;
- Minato/Mainnet configuration;
- retry/idempotency logic;
- API rate limits;
- transaction monitoring;
- backend authorization;
- audit logging;
- structured error handling;
- contract access control;
- contract deployment scripts;
- monitoring and alerts.

| The blockchain signing key must never be stored in frontend code.

19. Suggested Team Ownership

Product Manager

Responsible for:

- requirements;
- workflows;
- risk-policy definition;
- acceptance criteria;
- coordination with Soneium;
- CopySight/Soneium responsibility boundaries;
- MVP scope control.

Solution Architect

Responsible for:

- end-to-end architecture;
- Soneium architecture;
- EAS design;
- attestation schemas;
- trust model;
- signing architecture;
- API architecture;
- transaction lifecycle;
- data model;
- security decisions.

Smart Contract Developer

Responsible for:

- EAS schemas;
- resolver;
- Solidity;
- tests;
- deployment scripts;
- Minato deployment;
- Mainnet deployment;
- contract documentation.

CopySight Backend

Responsible for:

- existing validation API integration/support;
- blockchain metadata persistence;
- new internal APIs;
- blockchain status retrieval;

- frontend support APIs.

CopySight Frontend

Responsible for:

- registration UX;
- provenance UX;
- risk warning UX;
- transaction states;
- verification UI.

20. WBS Work Packages

Developers should estimate the following work packages separately.

WP1 — Discovery & Technical Design

- Soneium technical onboarding;
- Telegram technical coordination;
- CopySight API review;
- existing CopySight BE architecture review;
- registration workflow review;
- trust model;
- technical architecture;
- EAS vs custom-contract validation;
- signer model;
- risk-threshold definition;
- data model;
- final API contracts.

WP2 — Soneium Environment

- Minato configuration;
- RPC integration;
- development wallet;
- deployment wallet;
- signer infrastructure;
- faucet/test ETH;
- deployment pipeline;
- explorer integration;
- monitoring configuration.

WP3 — Smart Contracts / EAS

- Creation schema;
- Validation schema;
- CopySight Resolver;
- unit tests;
- integration tests;
- deployment scripts;
- Minato deployment;
- contract verification;
- documentation.

WP4 — CopySight Integration Service

- service skeleton;
- CopySight API adapter;
- asset hashing;

- result normalization;
- risk-policy module;
- blockchain client;
- transaction manager;
- idempotency;
- retry/error logic;
- signer integration.

WP5 — Proof of Creation

- registration API;
- CopySight validation;
- asset hash generation;
- EAS creation attestation;
- transaction persistence;
- blockchain confirmation;
- provenance retrieval;
- CopySight BE synchronization.

WP6 — IP Validation

- validation endpoint;
- CopySight API call;
- result normalization;
- configurable risk threshold;
- RISKY / NOT_RISKY response;
- validation attestation;
- registration gating integration.

WP7 — CopySight Backend

- blockchain DB fields;
- provenance APIs;
- transaction status;
- blockchain lookup;
- integration authentication;
- internal error handling.

WP8 — CopySight Frontend

- Register on Soneium;
- transaction states;
- provenance record;
- attestation details;
- risk warning;
- verification view.

WP9 — Testing

- CopySight API tests;

- smart-contract tests;
- integration tests;
- duplicate registration tests;
- invalid asset tests;
- API timeout tests;
- rate-limit tests;
- failed blockchain transaction tests;
- RPC outage tests;
- unauthorized signer tests;
- end-to-end Minato tests.

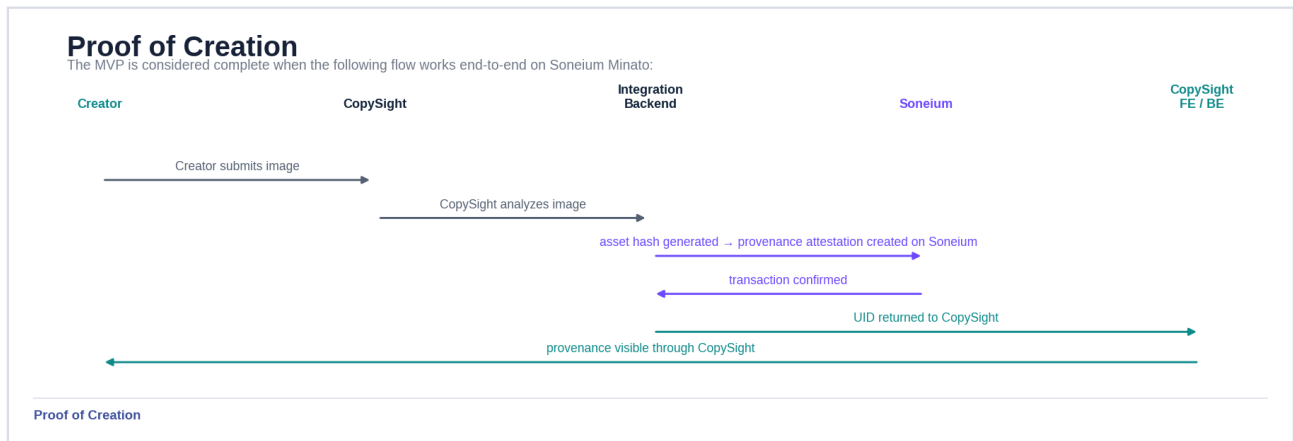
WP10 — Mainnet Readiness

- security review;
- signer security;
- RPC provider setup;
- configuration audit;
- production API keys;
- final Soneium technical review;
- Mainnet schemas/contracts;
- deployment;
- monitoring;
- runbook.

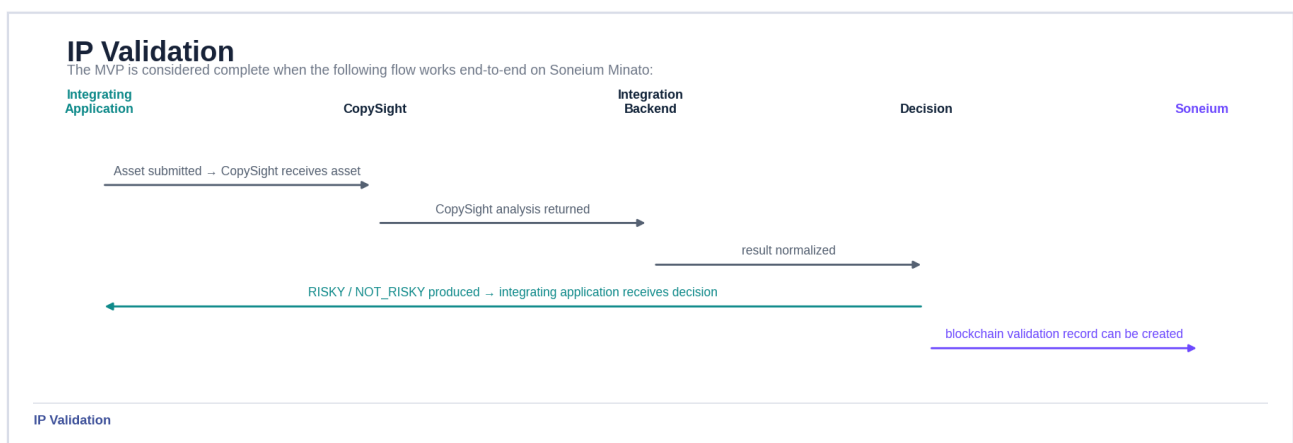
21. MVP Acceptance Criteria

The MVP is considered complete when the following flow works end-to-end on Soneium Minato:

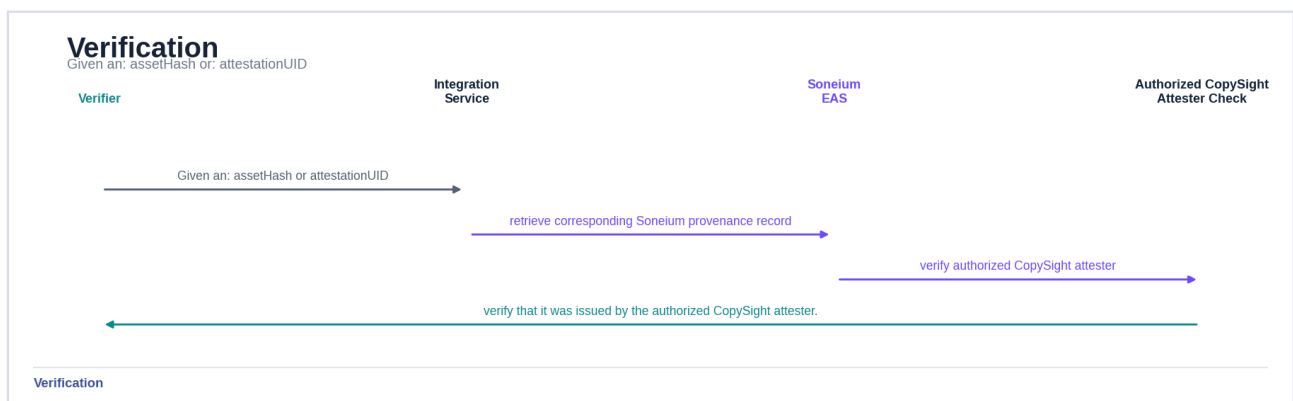
Proof of Creation



IP Validation



Verification



the system can retrieve the corresponding Soneium provenance record and verify that it was issued by the authorized CopySight attester.

22. Explicitly Out of Scope for MVP

The following should **not** be included in the initial WBS unless separately approved:

- redevelopment of CopySight AI/IP detection;
- creation of a new IP database;
- legal determination of copyright ownership;
- automatic copyright licensing;
- license-document verification;
- NFT marketplace;
- tokenization;
- royalty management;
- IP transfers;
- network-wide interception of arbitrary Soneium transactions;
- video production support unless CopySight API support is confirmed;
- 3D asset validation;
- decentralized file storage;
- storing original files directly on-chain;
- custom indexer unless performance requires it;
- account abstraction/gas sponsorship beyond basic integration.

Soneium does support account-abstraction tooling including gas sponsorship through supported providers, so gasless creator registration can be introduced later if desired rather than being a blocker for the first integration.

23. Decisions Required Before Final WBS Estimation

The architect/Product Manager should close the following items before developers finalize hours:

1. Which Soneium application/contract is the first consumer of the IP Validation API?
2. Does RISKY **block registration**, or only display a warning?
3. What similarity threshold generates RISKY?
4. Does CopySight already have an internal overall originality/risk score?
5. Is the creator required to connect an EVM wallet?
6. Who pays transaction gas?
7. Is CopySight the sole authorized attester?
8. Should creation attestations be revocable or permanently non-revocable?
9. Should validation attestations be stored for every analysis or only registrations?
10. Is a custom CopySightResolver required for MVP?
11. What CopySight data must be publicly visible versus represented only by hashes?
12. What transaction volume should the architecture support?
13. What Soneium RPC/provider will be used for production?

Once these decisions are closed, the above work packages can be directly converted into the engineering **WBS** → **roles** → **hours** → **dependencies** → **milestones**.